

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)» (МАИ)

Направление подготовки: 27.03.05 «Инноватика»

Практическая работа №6

по дисциплине «Структура программного обеспечения и базы данных»

Выполнил:

Студент гр. МЗО-236БВ-24

Зайцев Д.Ю.

Никандров И.А.

Карамян Ф.Э.

Преподаватель:

Александрова С.С.

Прудников А.В.

Москва 2025

Оглавление

Задание на практическую работу	2
Глава 3	3
3.6. Настроить возможность использовать 2 разные БД	5
4.3. Создать все варианты связей между моделями + нужное количество моделей для связей	7
4.7. Написать свои валидаторы	13
Вывод	16

Задание на практическую работу

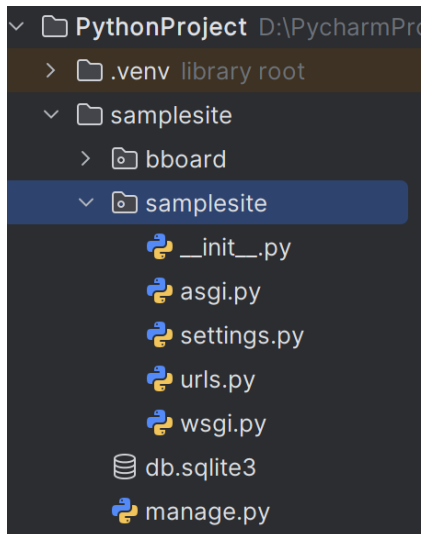
Цель работы: выполнить главу 3 и 4 из учебника по джанго

Задание: провести настройку существующего подключить БД sqlite3

Глава 3

1.Создание проекта

Мы уже создавали базовую файловую структуру проекта



2. В файле settings.py проверена конфигурация DATABASES. Для разработки используется стандартная СУБД SQLite3, не требующая установки дополнительных драйверов



3. Настройка основных параметров проекта (settings.py) согласно главе 3

Настроим язык и часовой пояс:

```

LANGUAGE_CODE = 'ru-ru'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

```

Всё параметры настроены верно еще с прошлой ЛР

4. Проверка `INSTALLED_APPS` (Убедиться, что приложение `bboard` добавлено в начале списка). Чтобы подключать шаблоны раньше, чем стандартные:

```

INSTALLED_APPS = [
    'bboard',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]

```

5. Улучшение модели `Bb` (согласно главе 4): В файле `bboard/models.py` обновить модель, добавив `__str__()` и, при необходимости, `ordering`

```

from django.db import models

class Bb(models.Model):
    title = models.CharField(max_length=50, verbose_name='Товар')
    content = models.TextField(verbose_name='Описание')
    price = models.FloatField(verbose_name='Цена')
    published = models.DateTimeField(auto_now_add=True, db_index=True, verbose_name='Опубликовано')
    rubric = models.ForeignKey(to='Rubric', null=True,
                              on_delete=models.PROTECT, verbose_name='Рубрика')

    class Meta:
        verbose_name_plural = 'Объявления'
        verbose_name = 'Объявление'
        ordering = ['-published']

class Rubric(models.Model):
    name = models.CharField(max_length=20, db_index=True, verbose_name='Название')

    def __str__(self):
        return self.name

    class Meta:
        verbose_name_plural = 'Рубрики'
        verbose_name = 'Рубрика'
        ordering = ['name']

```

В админке и сделает сортировку по дате публикации (новые -сверху)

6. Редактируем файл `bboard/admin.py`:

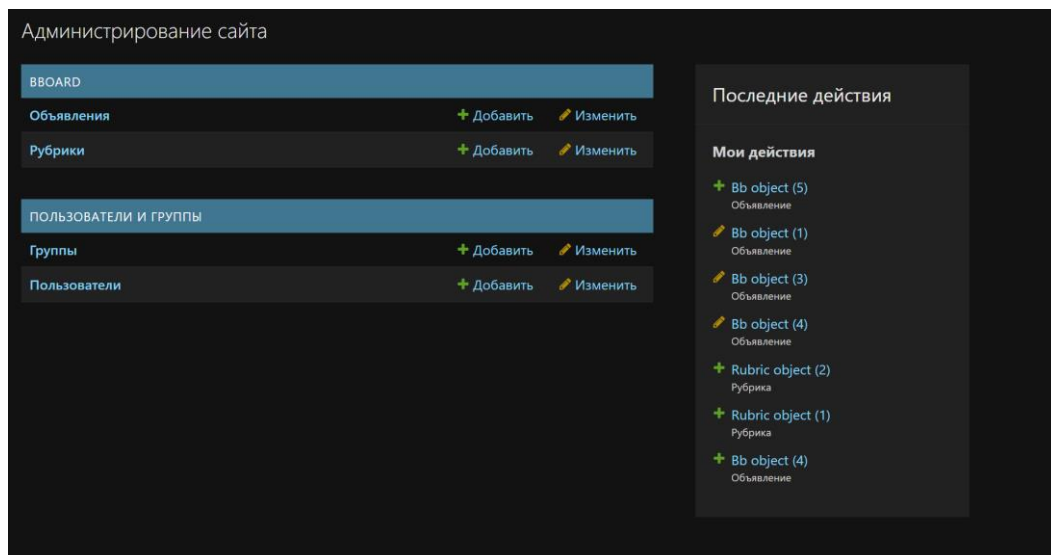
```

1 from django.contrib import admin
2
3 from .models import Bb,Rubric
4
5 class BbAdmin(admin.ModelAdmin):
6     list_display = ('rubric','title', 'content', 'price', 'published')
7     list_filter = ('published',)
8     list_display_links = ('title', 'content')
9     search_fields = ('title', 'content')
10
11 admin.site.register(Bb, BbAdmin)
12 admin.site.register(Rubric)
13

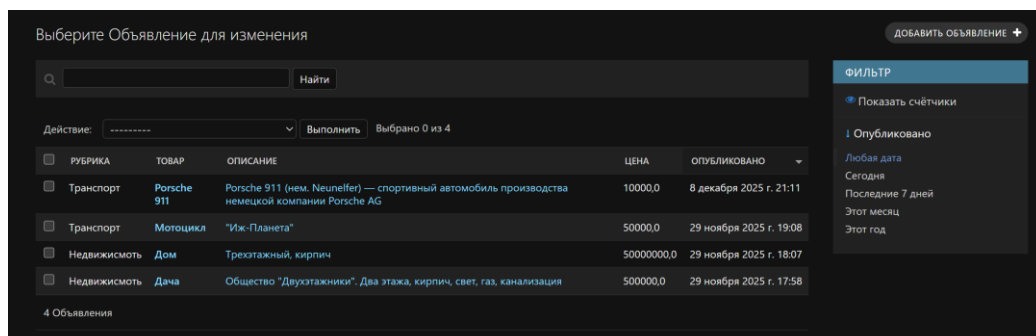
```

7.Проверка админки:

<http://127.0.0.1:8000/admin/>



8.Добавим объявления:

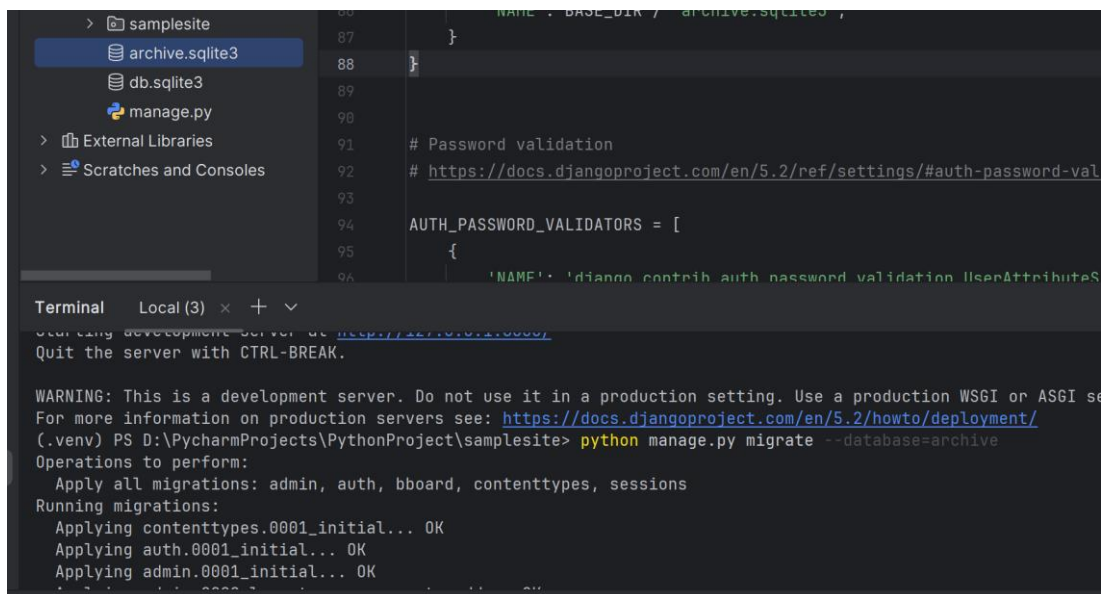


3.6. Настроить возможность использовать 2 разные БД

1. В файле settings.py изменен словарь DATABASES. Добавлена вторая база данных с псевдонимом archive, использующая СУБД SQLite

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    },  
    'archive': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'archive.sqlite3',  
    }  
}
```

2. Выполнена команда *python manage.py migrate --database=archive*. Это позволило создать физический файл базы данных archive.sqlite3 и синхронизировать структуру таблиц отдельно от основной базы



```
...  
    'NAME': BASE_DIR / 'archive.sqlite3',  
    }  
...  
# Password validation  
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators  
AUTH_PASSWORD_VALIDATORS = [  
    {  
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',  
    },  
    {  
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',  
    },  
    {  
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',  
    },  
    {  
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',  
    },  
]
```

```
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server. For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/.  
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py migrate --database=archive  
Operations to perform:  
  Apply all migrations: admin, auth, bboard, contenttypes, sessions  
Running migrations:  
  Applying contenttypes.0001_initial... OK  
  Applying auth.0001_initial... OK  
  Applying admin.0001_initial... OK
```

3. Через интерактивную консоль (shell) проведена проверка изоляции данных. С использованием метода *.using('archive')* создана запись в дополнительной БД. Проверка показала, что эта запись недоступна в основной базе данных default, что подтверждает корректную независимую

работу двух баз(в основной базе 4,потому что мы туда добавили 4 объявления)

```
(InteractiveConsole)
>>> from bboard.models import Bb
>>> Bb.objects.using('archive').create(title='Товар в архиве', price=100)
<Bb: Bb object (1)>
>>> print("В основной базе:", Bb.objects.all().count())
В основной базе: 4
>>> print("В архивной базе:", Bb.objects.using('archive').all().count())
В архивной базе: 1
>>>
```

4.3.Создать все варианты связей между моделями + нужное количество моделей для связей

1. В файл models.py добавлены новые сущности для реализации всех типов связей Django

Обоснование выбора моделей данных:

1. Модель **Rubric** (ForeignKey): Использована связь «Один-ко-многим», так как каждое объявление должно строго относиться только к одной тематической категории
2. Модель **Sticker** (ManyToMany): Использована связь «Многие-ко-многим», чтобы реализовать систему тегов: одно объявление может иметь несколько меток одновременно (например, «Срочно» и «Торг»)
3. Модель **BbExtra** (OneToOne): Использована связь «Один-к-одному» для декомпозиции данных. Редко используемая или техническая информация вынесена в отдельную таблицу, чтобы не перегружать основную модель

```

18 class Rubric(models.Model): 7 usages
19     name = models.CharField(max_length=20, db_index=True, verbose_name='Название')
20
21     def __str__(self):
22         return self.name
23
24     class Meta:
25         verbose_name_plural = 'Рубрики'
26         verbose_name = 'Рубрика'
27         ordering = ['name']
28
29 class Sticker(models.Model): 2 usages
30     name = models.CharField(max_length=20, verbose_name='Стикер')
31
32     def __str__(self):
33         return self.name
34
35     class Meta:
36         verbose_name = 'Стикер'
37         verbose_name_plural = 'Стикеры'
38
39
40 class BbExtra(models.Model): 2 usages
41     bb = models.OneToOneField(Bb, on_delete=models.CASCADE, primary_key=True, verbose_name='Объявление')
42
43     is_broken = models.BooleanField(default=False, verbose_name='Требуется ремонт?')
44     contact_email = models.EmailField(blank=True, verbose_name='Email для связи')
45
46     class Meta:
47         verbose_name = 'Дополнительно'
48         verbose_name_plural = 'Дополнительные параметры'

```

2. Все новые модели зарегистрированы в файле admin.py, что позволяет управлять ими через стандартный интерфейс администратора

```

1 from django.contrib import admin
2 from .models import Bb, Rubric, Sticker, BbExtra
3
4 class BbAdmin(admin.ModelAdmin): 1 usage
5     list_display = ('title', 'rubric', 'content', 'price', 'published')
6     list_filter = ('published', 'rubric')
7     list_display_links = ('title', 'content')
8     search_fields = ('title', 'content')
9
10 admin.site.register(Bb, BbAdmin)
11 admin.site.register(Rubric)
12 admin.site.register(Sticker)
13 admin.site.register(BbExtra)

```

3. Команда makemigrations сформировала файл миграции с инструкциями для создания новых моделей BbExtra, Sticker и добавления поля stickers

```

(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py makemigrations
Migrations for 'bboard':
  bboard\migrations\0003_bbextra_sticker_bb_stickers.py
    + Create model BbExtra
    + Create model Sticker
    + Add field stickers to bb
(.venv) PS D:\PycharmProjects\PythonProject\samplesite>

```


4. Команда migrate выполнила эти инструкции, физически создав новые таблицы и связи в базе данных sqlite3

```
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bboard, contenttypes, sessions
Running migrations:
  Applying bboard.0003_bbextra_sticker_bb_stickers... OK
(.venv) PS D:\PycharmProjects\PythonProject\samplesite>
```

5. Проверим как все методы работают:

- 1) Поле «Рубрика» (ForeignKey): отображается как выпадающий список (SelectBox). Это ограничивает ввод, позволяя выбрать строго одну категорию из существующих

The screenshot displays a web application interface. At the top, there's a header with a search bar and navigation links. Below the header, a table lists items for sale. The table has columns: TOVAR (Item), Рубрика (Category), ОПИСАНИЕ (Description), ЦЕНА (Price), ОПУБЛИКОВАНО (Published), and МЕТКИ (Tags). The items listed are: Квартира на выхино, Дом, Дача, Porsche 911, and Мотоцикл. Below the table, there's a section for filters. On the left, a sidebar contains links to different sections: ВВОARD, Дополнительные параметры, Объявления, Рубрики, and Стикеры. On the right, there's a section titled 'Выберите Рубрика для изменения' (Select Category for change) with a dropdown menu and a list of categories: Рубрика, Недвижимость, and Транспорт.

ТОВАР	Рубрика	ОПИСАНИЕ	ЦЕНА	ОПУБЛИКОВАНО	МЕТКИ
☐ Квартира на выхино	Недвижимость	Продам,пожалуйста	5000,0	13 декабря 2025 г. 20:51	Срочно, Быстро
☐ Дом	Недвижимость	Трехэтажный, кирпич	50000000,0	29 ноября 2025 г. 18:07	-
☐ Дача	Недвижимость	Общество "Двухэтажки". Два этажа, кирпич, свет, газ, канализация	500000,0	29 ноября 2025 г. 17:58	-
☐ Porsche 911	Транспорт	Porsche 911 (нем. Neunelfer) — спортивный автомобиль производства немецкой компании Porsche AG	10000,0	8 декабря 2025 г. 21:11	-
☐ Мотоцикл	Транспорт	"Иж-Планета"	50000,0	29 ноября 2025 г. 19:08	-

5 Объявления

Начните печатать для фильтрации...

ВВОARD

- Дополнительные параметры + Добавить
- Объявления + Добавить
- Рубрики + Добавить
- Стикеры + Добавить

ПОЛЬЗОВАТЕЛИ И ГРУППЫ

- Группы + Добавить
- Пользователи + Добавить

Выберите Рубрика для изменения

Действие: ----- Выполнить Выбрано 0 из 2

- ☐ Рубрика
- ☐ Недвижимость
- ☐ Транспорт

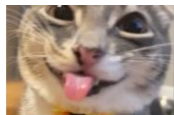
2 Рубрики

```

1 {% extends "layout/basic.html" %}
2 {% block title %}{ current_rubric.name }{% endblock %}
3 {% block content %}
4
5 <h2>Рубрика: {{ current_rubric.name }}</h2>
6 for bb in bbs %}
7 <div>
8   <h2>{{ bb.title }}</h2>
9   <p>{{ bb.content }}</p>
10   <p>
11     {% if bb.stickers.all %}
12     <strong>Метки:</strong>
13     {% for sticker in bb.stickers.all %}
14       <span style="background-color: #eee; padding: 2px 5px; margin-right: 5px; border-radius: 3px;">
15         {{ sticker.name }}
16       </span>
17     {% endfor %}
18   {% endif %}
19   </p>
20   <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
21 </div>
22 {% endfor %}
23 {% endblock %}

```

Вот так отображается



ОИ

[Главная](#)

[Добавить](#)

[Недвижимость](#)

[Транспорт](#)

Квартира на выхино

Продам,пожалуйста

[Недвижимость](#)

Метки:

13.12.2025 20:51:06

Porsche 911

- 2) Поле «Стикеры» (ManyToManyField): Отображается как виджет множественного выбора. Это позволяет присвоить одному объявлению любой набор меток (тегов), созданных ранее

```
admin.py × settings.py models.py <> index.html <> rubric_bbs.html <> bb_create.html
1 from django.contrib import admin
2 from .models import Bb, Rubric, Sticker, BbExtra
3
4 class BbAdmin(admin.ModelAdmin): 1 usage
5     list_display = ('title', 'rubric', 'content', 'price', 'published', 'get_stickers')
6     list_filter = ('published', 'rubric', 'stickers')
7     list_display_links = ('title', 'content')
8     search_fields = ('title', 'content')
9
10     def get_stickers(self, obj): 1 usage
11         return ", ".join([s.name for s in obj.stickers.all()])
12
13     get_stickers.short_description = 'Метки'
14
15 admin.site.register(Bb, BbAdmin)
16 admin.site.register(Rubric)
17 admin.site.register(Sticker)
18 admin.site.register(BbExtra)
```

Добавили в admin.py метки чтоб они отображались и в index.html

```
admin.py settings.py models.py <> index.html × <> rubric_bbs.html <> bb_create.html
1 {% extends "layout/basic.html" %}
2 {% block content %}
3 {% for bb in bbs %}
4 <div>
5     <h2>{{ bb.title }}</h2>
6     <p>{{ bb.content }}</p>
7     <p><a href="{% url 'rubric_bbs' bb.rubric.pk %}">
8         {{ bb.rubric.name }}</a></p>
9     <p>
10         {% if bb.stickers.all %}
11             <strong>Метки:</strong>
12             {% for sticker in bb.stickers.all %}
13                 <span style="background-color: #eee; padding: 2px 5px; margin-right: 5px; border-radius: 3px;">
14                     {{ sticker.name }}
15                 </span>
16             {% endfor %}
17         {% endif %}
18     </p>
19     <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
20 </div>
21 {% endfor %}
22 {% endblock %}
```

Так отображается в админке

Действие:

▼

Выполнить

Выбрано 0 из 3

☐ СТИКЕР

☐ Быстро

☐ Торг

☐ Срочно

3 Стикеры

Действие:

▼

Выполнить

Выбрано 0 из 5

5 Объявления

Опу

Любая

Сегод

После

Этот м

Этот г

Рубр

Все

Недви

Транс

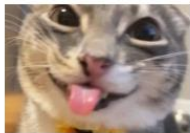
-

Сти

Все

Срочн

А так на доске



[Главная](#)

[Добавить](#)

[Недвижимость](#)

[Транспорт](#)

Рубрика: Недвижимость

Квартира на выхино

Продам,пожалуйста

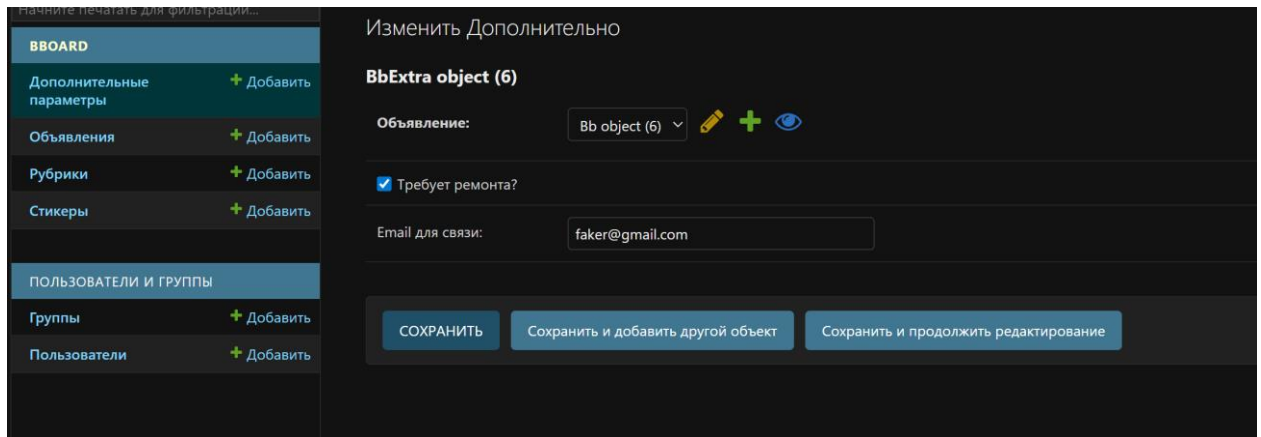
Метки:

Срочно

Быстро

13.12.2025 20:51:06

- 3) Модель BbExtra (OneToOne): Использована связь «Один-к-одному» для декомпозиции данных. Редко используемая или техническая информация вынесена в отдельную таблицу, чтобы не перегружать основную модель



4.7. Написать свои валидаторы

Пишем свои валидаторы

- Валидатор цены (validate_non_negative): запрещает отрицательные значения, исключая ошибки ввода стоимости
- Валидатор спецсимволов (validate_clean_title): блокирует знаки !, @, # в названии, обеспечивая визуальную чистоту списка
- Грамматический валидатор (validate_capital_start): Принудительно требует начинать заголовок с заглавной буквы. Это повышает культуру оформления объявлений и делает список товаров единообразным и аккуратным

```
def validate_non_negative(value): 2 usages (1 dynamic)
    if value < 0:
        raise ValidationError('Цена не может быть меньше нуля!')

def validate_clean_title(value): 2 usages (1 dynamic)
    forbidden_chars = '!@#$$%^&*_=+'
    for char in value:
        if char in forbidden_chars:
            raise ValidationError(f'Символ "{char}" запрещен. Пишите название словами.')

def validate_capital_start(value): 1 usage
    if value and not value[0].isupper():
        raise ValidationError('Заголовок должен начинаться с большой буквы (например, "Зил", а не "зил").')
```

Выполним миграцию для актуализации схемы базы данных после

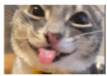
редактирования файла `bboard/models.py`, в котором к полям модели были подключены новые функции валидации

```
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py makemigrations
Migrations for 'bboard':
  bboard\migrations\0004_alter_bb_price_alter_bb_title.py
    ~ Alter field price on bb
    ~ Alter field title on bb
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> 
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bboard, contenttypes, sessions
Running migrations:
  Applying bboard.0004_alter_bb_price_alter_bb_title... OK
(.venv) PS D:\PycharmProjects\PythonProject\samplesite>
```

Еще мы добавили в `forms.py` 'stickers', для того чтобы они у нас выводили на главном экране



```
1 > import ...
4
5 class BbForm(ModelForm): 2 usages
6     class Meta:
7         model = Bb
8         fields = ('title', 'content', 'price', 'rubric', 'stickers')
```



ОБЪЯВЛЕНИЯ

[Главная](#)

[Добавить](#)

[Недвижимость](#)

[Транспорт](#)

Добавление объявления

Товар:

Описание:

Цена:

Рубрика:

Стикеры:

Теперь проверим наши валидаторы:

- 1) Цена не может быть отрицательной

Добавление объявления

Товар:

Описание:

уу

- Цена не может быть меньше нуля!

Цена:

Рубрика:

Стикерс:

Срочно ▲
Торг
Быстро ▼

2) Блокировка символом вместо слов

Добавление объявления

- Символ "!" запрещен. Пишите название словами.

Товар:

Описание:

уу

Цена:

Рубрика:

Стикерс:

Срочно ▲
Торг
Быстро ▼

3) Нельзя с маленькой буквы

Добавление объявления

- Заголовок должен начинаться с большой буквы (например, "Зил", а не "зил").

Товар:

Описание:

уу

Цена:

Рубрика:

Стикеры:

Срочно ▲

Торг

Быстро ▼

Если всё правильно написали, то валидаторы не сработают и объявление у нас появится на доске

Зил

уу

Транспорт

Метки: Срочно

13.12.2025 22:57:25

☐	ТОВАР	РУБРИКА	ОПИСАНИЕ	ЦЕНА	ОПУБЛИКОВАНО	МЕТКИ
☐	Зил	Транспорт	уу	500,0	13 декабря 2025 г. 22:57	Срочно
☐	Квартира на	Недвижимость	Продам пожалуйста	5000,0	13 декабря 2025 г. 20:51	Срочно

Вывод

В ходе выполнения заданий было доработано веб-приложение на Django

Основные результаты:

1. **Структура данных:** Реализована связь «Многие-ко-многим» для системы стикеров (меток)
2. **Интерфейс:** Настроены шаблоны для отображения меток и обновлена форма добавления объявления (forms.py) для их выбора
3. **Безопасность данных:** Разработаны и подключены пользовательские валидаторы, которые автоматически блокируют некорректный ввод (отрицательная цена, ошибки в заголовке)
4. **База данных:** Отработано управление миграциями для внесения изменений в структуру БД

Итог: Функционал сайта расширен, а введенная система валидации защищает приложение от логических ошибок и некорректных данных